

Processes, Tasks and Agents — A Readable Companion to the Elixir Docs

Chapter 3 · checked against Elixir 1.20 · examples run on Elixir 1.14 · Sep 26, 2026

1. Why Elixir gives you abstractions over processes

Every concurrent thing in Elixir is a BEAM process: isolated, cheap, communicating only by messages (Erlang book, chapter 2). You can work with them directly through `spawn`, `send` and `receive`, but the Elixir guide steers you away from that: “Tasks build on top of the spawn functions to provide better error reports and introspection.”

Elixir’s standard library gives three abstractions, each shaped for one job:

- **Task:** run a computation concurrently, often to get a result back.
- **Agent:** hold a piece of state that several processes read and update.
- **GenServer:** a long-lived process with its own protocol (chapter 4).

They are thin, and that’s the point: each keeps the underlying process semantics visible. The mistakes in this chapter come from forgetting those semantics: links that propagate crashes, laziness that means nothing runs, functions that execute in a different process from the one that wrote them.

2. The building blocks

Task

Function	Linked to caller?	Result	Use for
<code>Task.async/1</code> + <code>await/2</code>	Yes, and monitored	Must be awaited, default 5000 ms	Concurrent part of the caller’s own work
<code>Task.await_many/2</code>	—	Results in task order	Several <code>async</code> tasks at once
<code>Task.async_stream/3</code>	Yes	Lazy stream of <code>{:ok, v}</code> / <code>{:exit, r}</code>	Bounded concurrency over a collection
<code>Task.start_link/1</code>	Yes	None	Fire-and-forget inside a supervision tree
<code>Task.Supervisor.async_nolink/3</code>	No, monitored only	<code>{ref, result}</code> message or <code>:DOWN</code>	Work whose failure must not crash the caller
<code>Task.Supervisor.start_child/2</code>	No	None	Supervised fire-and-forget

`async_stream` defaults worth knowing: `max_concurrency` equals the number of online schedulers, results are ordered, each item has a 5000 ms `timeout`, and `on_timeout: :exit` makes a timeout exit the *caller*.

Agent

An Agent is a process holding one value. `Agent.get/2`, `update/2` and `get_and_update/2` take a function, and the docs are explicit about where it runs: “the functions passed as arguments to the calls to Agent functions are invoked inside the agent (the server).”

When to use which

Need	Use
Speed up independent work, wait for results	<code>Task.async_stream</code> or <code>async</code> + <code>await_many</code>
Work that may fail without taking you down	<code>Task.Supervisor.async_nolink</code>
Background work nobody waits for	<code>Task.Supervisor.start_child</code>
Small shared state with simple updates	Agent
State with a protocol, timers or complex rules	<code>GenServer</code> (chapter 4)
Read-mostly shared data	ETS or <code>:persistent_term</code> , not a process (Erlang book, chapter 4)

3. How it actually works

async links both ways

`Task.async` links the task to the caller. If the task crashes, the caller crashes; if the caller crashes, the task dies. Only the caller can await the result, and it must, because the reply message is always sent.

Streams are lazy

`Task.async_stream/3` returns a stream. Nothing runs until the stream is consumed by `Enum.to_list/1`, `Stream.run/1` or another `Enum` function.

Functions run where they’re sent

A function passed to an Agent runs inside the Agent’s process. A closure passed to `Task.async` runs in the task’s process, and everything it references is *copied* there (Erlang book, chapter 2). The process boundary is invisible in the syntax but decides both performance and atomicity.

Reads and writes are separate messages

`Agent.get` and `Agent.update` are two separate calls. Between them, any number of other processes can update the agent. Only a single call (`update/2` with a function, or `get_and_update/2`) is atomic.

4. Plausible but wrong

4.1 Unbounded parallelism

```
urls
|> Enum.map(fn url -> Task.async(fn -> HTTP.get(url) end) end)
|> Task.await_many(30_000)
```

With 50,000 URLs this starts 50,000 requests at once: exhausted connection pools, file descriptors and remote rate limits. It works in the demo with ten URLs. Fix: `Task.async_stream(urls, &HTTP.get/1, max_concurrency: 20, timeout: 30_000)` for concurrency with a limit.

4.2 One slow item kills the whole batch

```
results =
  items
  |> Task.async_stream(&process/1)
  |> Enum.to_list()
```

With the defaults, any item taking longer than 5 seconds makes the *caller* exit with `:timeout`, discarding every result already computed. Fix: set an explicit `timeout`, add `on_timeout: :kill_task`, and handle the resulting `{:exit, :timeout}` entries:

```
|> Task.async_stream(&process/1, timeout: 30_000, on_timeout: :kill_task)
|> Enum.flat_map(fn {:ok, r} -> [r]; {:exit, _} -> [] end)
```

4.3 A stream that never runs

```
def notify_all(users) do
  Task.async_stream(users, &Mailer.send_notice/1, max_concurrency: 10)
  :ok
end
```

This returns `:ok` and sends nothing. The stream was built and thrown away. Tests that only check the return value pass. Fix: consume it with `|> Stream.run()`, or `|> Enum.to_list()` if you need the results.

4.4 Read, then update an Agent

```
def increment(counter) do
  value = Agent.get(counter, &&1)
  Agent.update(counter, fn _ -> value + 1 end)
end
```

Other processes can update between `get` and `update`, and their changes get overwritten. In a test with 100 concurrent increments this ended with a count of `1`. Fix: do the whole change in one call: `Agent.update(counter, &(&1 + 1))`, or `Agent.get_and_update/2` when you also need the old value.

4.5 Expensive work inside the Agent

```
Agent.get(Stats, fn events -> Statistics.percentiles(events) end)
```

`percentiles/1` runs inside the Agent, so every other reader and writer waits behind it. Fix: take the data out (`Agent.get(Stats, &1)`) and compute in the caller. If most access is reads, store the data in ETS instead.

4.6 A closure that copies more than it uses

```
def handle_call({:export, id}, _from, state) do
  Task.start(fn -> Exporter.run(id, state.settings.format) end)
  {:reply, :ok, state}
end
```

The closure references `state`, so the entire server state (maybe megabytes) is copied into the new process just to read one field. The process anti-patterns guide calls this *sending unnecessary data*. Fix: extract what you need first: `format = state.settings.format`, then use `format` in the closure.

4.7 Unsupervised background work

```
def handle_cast({:reindex, id}, state) do
  Task.start(fn -> Search.reindex(id) end)
  {:noreply, state}
end
```

The task belongs to no supervision tree. At shutdown it is killed mid-write rather than stopped in order. When it crashes, nothing notices, and you can't list or limit these tasks. The process anti-patterns guide names this *unsupervised processes*. Fix: `Task.Supervisor.start_child(MyApp.TaskSup, fn -> ... end)` with the task supervisor in your application's tree.

4.8 `async_nolink` without handling its messages

```
def handle_call(:refresh, _from, state) do
  Task.Supervisor.async_nolink(MyApp.TaskSup, &fetch_rates/0)
  {:reply, :ok, state}
end
```

The task's result arrives later as a `{ref, result}` message, and a `{:DOWN, ref, ...}` message follows (or comes alone if the task crashed). Without `handle_info` clauses for them, `GenServer`'s default logs "unexpected message" warnings, and the result is lost. Fix: keep `task.ref` in the state and handle both messages:

```
def handle_info({ref, rates}, %{task_ref: ref} = s) do
  Process.demonitor(ref, [:flush])
  {:noreply, %{s | rates: rates, task_ref: nil}}
end
```

```
def handle_info({:DOWN, ref, :process, _, _reason}, %{task_ref: ref} = s),  
  do: {:noreply, %{s | task_ref: nil}}
```

5. Review checklist and sources

- ☐ Concurrency over collections is bounded (`async_stream` with `max_concurrency`)
- ☐ `async_stream` has an explicit `timeout`; slow items use `on_timeout: :kill_task` where partial results are acceptable
- ☐ Every stream is consumed (`Stream.run/1` or an `Enum` function)
- ☐ Agent updates are single calls; no `get` followed by `update`
- ☐ No expensive functions run inside Agents
- ☐ Closures sent to other processes capture only the data they need
- ☐ Background work runs under a `Task.Supervisor`
- ☐ `async_nolink` results and `:DOWN` messages are handled, with `demonitor(ref, [:flush])`
- ☐ `Task.async` is used only where a task crash should crash the caller

Sources

- Processes guide: <https://hexdocs.pm/elixir/processes.html>
- Task: <https://hexdocs.pm/elixir/Task.html>
- Task.Supervisor: <https://hexdocs.pm/elixir/Task.Supervisor.html>
- Agent: <https://hexdocs.pm/elixir/Agent.html>
- Process-related anti-patterns: <https://hexdocs.pm/elixir/process-anti-patterns.html>